MACHINE LEARNING WEEK 6

LAB REPORT

Nevin Mathew Thomas

PES2UG23CS381

Course: UE23CS352A

16-09-2025

Introduction

Purpose of the lab: To gain hands-on experience by implementing a neural network from scratch without high-level frameworks like TensorFlow or PyTorch.

Tasks performed:

- Generating a custom dataset.
- Implementing core neural network components: activation functions, loss functions, forward propagation, backpropagation, and weight updates.
- Training a neural network to approximate the generated polynomial curve.
- Evaluating and visualizing the model's performance.

Dataset Description

Polynomial Type: CUBIC: $y = 2.21x^3 + -0.58x^2 + 3.79x + 10.13$

Noise Level: $\varepsilon \sim N(0, 1.83)$

Architecture: Input(1) → Hidden(32) → Hidden(72) → Output(1)

Learning Rate: 0.005

Architecture Type: Narrow-to-Wide Architecture

Each dataset contains 100,000 samples. The data is split into a training set (80%) and a testing set (20%). Both input (x) and output (y) are standardized.

Methodology

Key Concepts Implemented:

Activation Functions: Implemented the Rectified Linear Unit (ReLU), which introduces non-linearity and helps mitigate the vanishing gradient problem.

Loss Function: The Mean Squared Error (MSE) is used to measure how well the network's predictions align with the true values.

Forward Propagation: This process involves passing input data sequentially through each layer to produce a final prediction.

Backpropagation: This method computes the gradient of the loss with respect to each parameter using the chain rule to update weights and biases.

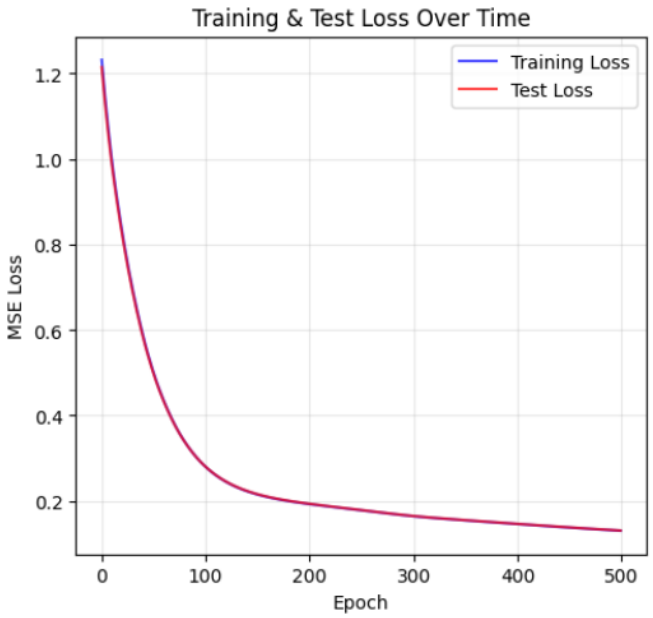
Gradient Descent: Parameters are updated in the opposite direction of the gradient, scaled by a learning rate, to gradually reduce the loss.

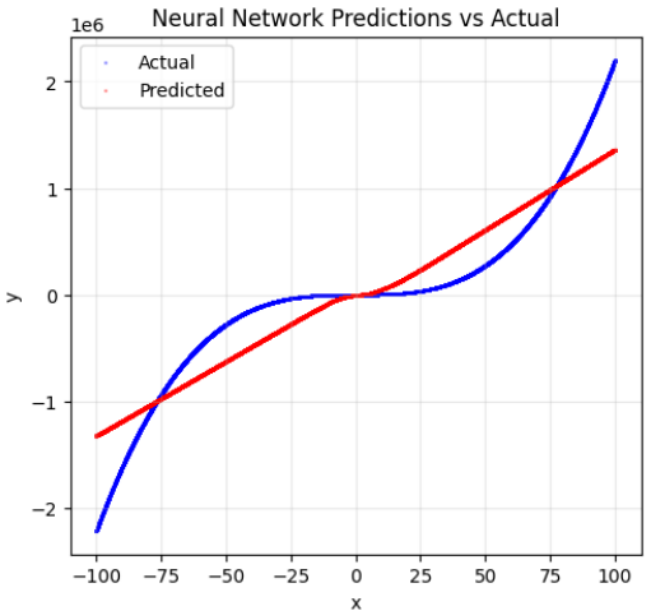
Results and Analysis

Experiment	Learning Rate	No. of epochs	Optimizer	Activation function	Training Loss	Test Loss	R2 score	Observation
Baseline	0.005	500	Gradient Descent	ReLU	0.13062	0.13094	0.8699	The model shows good initial performance with a low test loss, indicating it is not overfitting.
1	0.007	600	Gradient Descent	ReLU	0.09178	0.09196	0.9087	A slight increase in learning rate and epochs significantly improves the model's accuracy.
2	0.007	700	Gradient Descent	ReLU	0.07978	0.07997	0.9206	More epochs at the same learning rate lead to further improvements, as the model continues to learn and converge.
3	0.01	700	Gradient Descent	ReLU	0.05300	0.05319	0.9472	A higher learning rate results in a substantial jump in performance, showing the model can find a better solution faster.
4	0.02	800	Gradient Descent	ReLU	0.01248	0.01268	0.9874	The combination of the highest learning rate and number of epochs yields the best performance, as the model reaches an optimal solution with very low error.

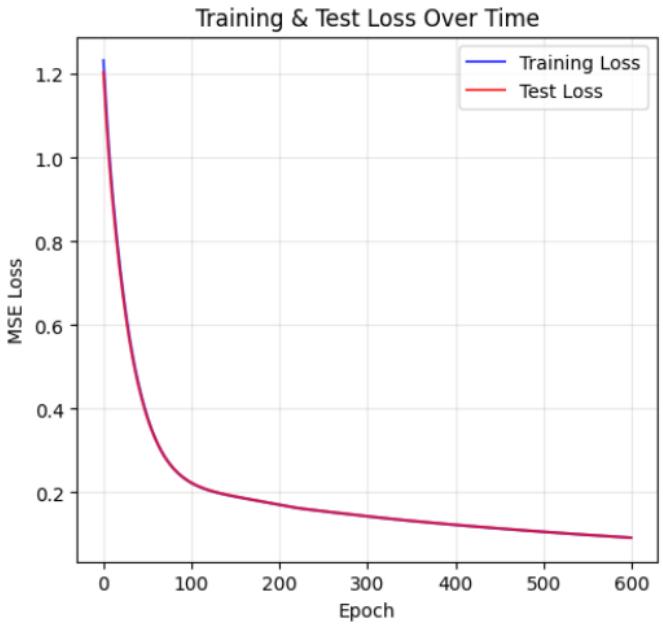
Plots

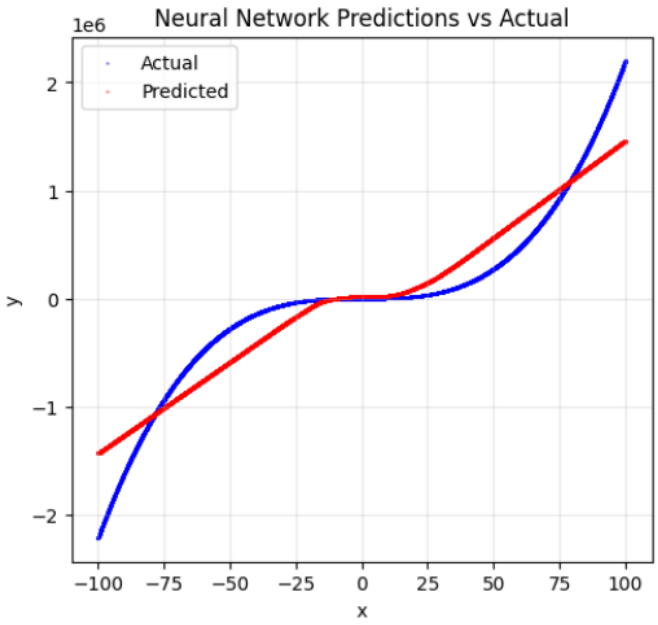
Baseline:



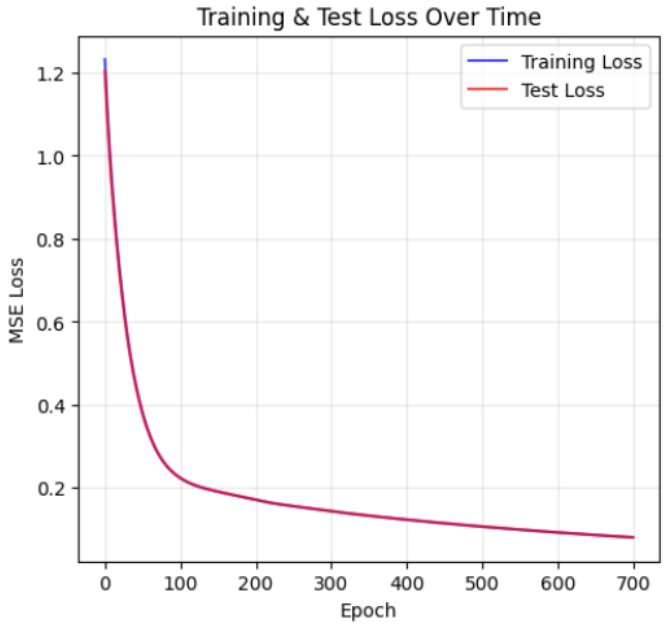


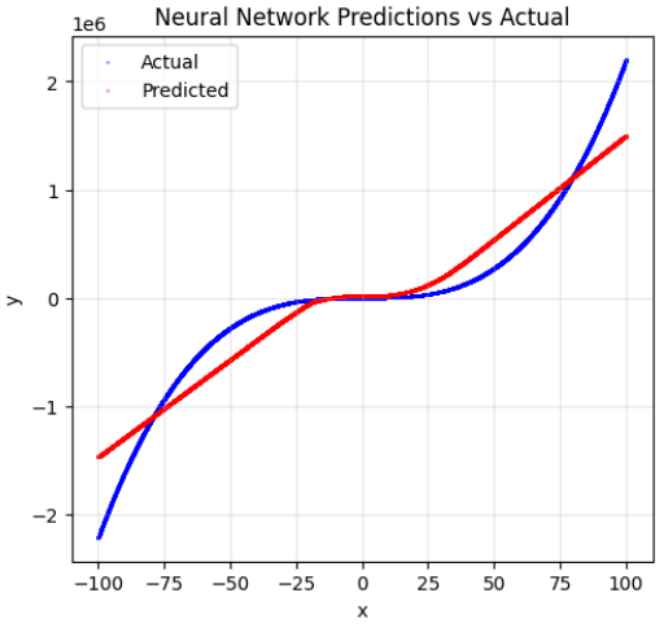
Experiment 1:



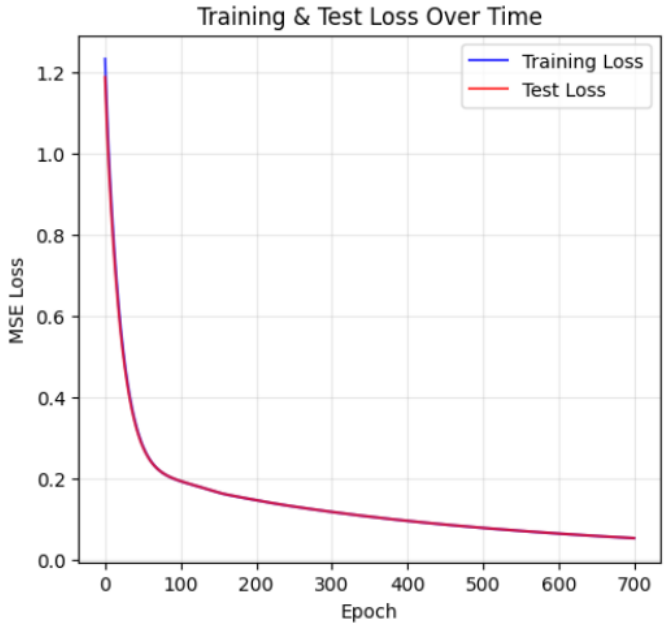


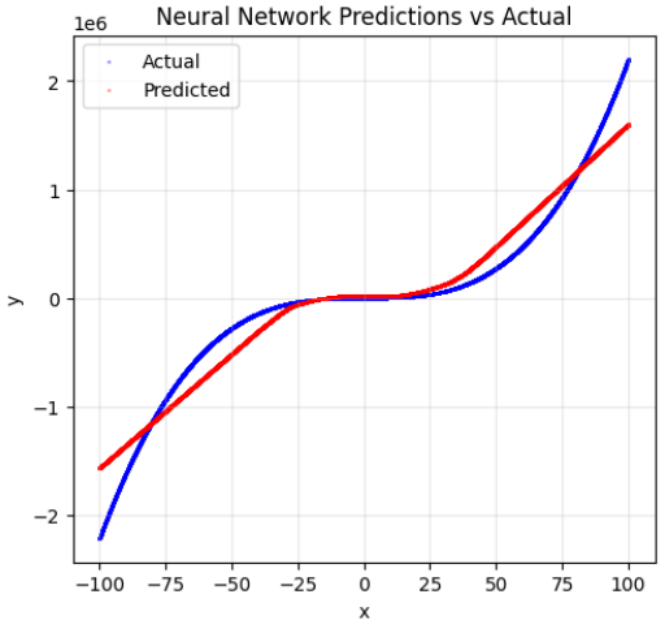
Experiment 2:





Experiment 3:





Experiment 4: